

Constructors:

Constructor in C++ is a special method that is invoked automatically at the time of object creation. It is used to initialize the data members of new objects generally. The constructor in C++ has the same name as the class. It constructs the values i.e. provides data for the object which is why it is known as a constructor.

- Constructor is a member function of a class, whose name is the same as the class name.
- Constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object that is why it is known as a constructor.
- Constructors do not return value, hence they do not have a return type.
- A constructor gets called automatically when we create the object of the class.
- Constructors can be overloaded.

Syntax:

```
class-name (list-of-parameters)
{
    // constructor definition
}
```

```
#include <iostream>
using namespace std;

class Teacher
{
private:
    double salary;

public:
    string name;
```

```
string dept;
string subject;

Teacher()
{
    dept = "Computer science";
}

void setSalary(double s)
{
    salary = s;
}

double getSalary()
{
    return salary;
}
};

int main()
{
    Teacher t1;
    t1.setSalary(25000);
    t1.name = "aadil";
    t1.subject = "C++";

    cout << t1.dept << endl;

    return 0;
}
```

Characteristics of Constructors in C++

The following are some main characteristics of the constructors in C++:

- The name of the constructor is the same as its class name.
- Constructors are mostly declared in the public section of the class though they can be declared in the private section of the class.
- A constructor gets called automatically when we create the object of the class.
- Constructors can be overloaded.
- A constructor cannot be inherited.
- The addresses of the Constructor cannot be referred to.
- The constructor makes implicit calls to **new** and **delete** operators during memory allocation.

Types of Constructor in C++

1. Default Constructor in C++

A default constructor is a constructor that doesn't take any argument. It has no parameters. It is also called a zero-argument constructor.

Syntax of Default Constructor

```
className(){  
    //body_of_constructor  
}
```

```
class student  
{  
public:  
    string name;  
    int rollNumber;  
  
    student(){  
        name = "Unknown";  
        rollNumber = 0000;  
    }  
}
```

```

    student(string name, int rollNumber){
        this->name = name;
        this->rollNumber = rollNumber;
    }

    void getInfo(){
        cout << "Name : " << name << endl;
        cout << "RollNumber : " << rollNumber << endl;
    }
};

int main(){
    student s1;
    student s2("Aadil", 2222);
    s1.getInfo();
    s2.getInfo();

    return 0;
}

```

2. Parameterized Constructor in C++

Parameterized Constructors make it possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

Syntax of Parameterized Constructor

```

className (parameters...){
    // body
}

```

```

#include <iostream>
using namespace std;

```

```
class Teacher
{
private:
    double salary;

public:
    string name;
    string dept;
    string subject;

    Teacher(string n, string d, string s, double sal)
    {
        name = n;
        dept = d;
        subject = s;
        salary = sal;
    }

    void getDetails()
    {
        cout << "Name : " << name << endl;
        cout << "Dept : " << dept << endl;
        cout << "Salary : " << salary << endl;
    }
};

int main()
{
    Teacher t1("Aadil", "ComputerScience", "C++", 25000);
    t1.getDetails();
}
```

```
return 0;  
}
```

When an object is declared in a parameterized constructor, the initial values have to be passed as arguments to the constructor function.

3.Copy Constructor:

A copy constructor in C++ is a specialized constructor used to create a new object as a copy of an existing object. It plays a crucial role in ensuring that objects are copied correctly, particularly in cases where objects manage dynamic resources like memory.

The compiler provides a default copy constructor to all the classes.

Syntax:

```
className (className &obj){  
    ....  
}
```

Example:

```
#include <iostream>  
using namespace std;  
  
class Teacher  
{  
private:  
    double salary;  
  
public:  
    string name;  
    string dept;
```

```
string subject;

// parameterized constructor
Teacher(string n, string d, string s, double sal)
{
    name = n;
    dept = d;
    subject = s;
    salary = sal;
}

// copy constructor
Teacher(Teacher &origObj) // pass by reference
{
    this->name = origObj.name;
    this->dept = origObj.dept;
    this->subject = origObj.subject;
    this->salary = origObj.salary;
}

void getDetails()
{
    cout << "Name : " << name << endl;
    cout << "Dept : " << dept << endl;
    cout << "Salary : " << salary << endl;
}
};

int main()
{
    Teacher t1("Aadil", "ComputerScience", "C++", 25000);
```

```
Teacher t2(t1); // custom copy constructor  
t2.getDetails();  
  
return 0;  
}
```

There are two types of copy constructors which are –

- Default Copy Constructors (Does Shallow Copying)
- User Defined Copy Constructors (Does Deep Copying)

Shallow Copy

When we do not create our own copy constructor. **C++ compiler will create a default copy constructor** for each class which does a member-wise copy between objects.

This default copy constructor does shallow copy.

A shallow copy of an object copies all the member values from one object to another.

In normal cases shallow copy is ok, but the issue arises when we do dynamic memory allocation.

Deep Copy:

A deep copy on the other hand , not only copies the member values but also makes copy of any dynamically allocated memory that the member points to.

Constructor Overloading in C++

In C++, We can have more than one constructor in a class with same name, as long as each has a different list of arguments. This concept is known as Constructor Overloading and is quite similar to function overloading.

- Overloaded constructors essentially have the same name (exact name of the class) and different by number and type of arguments.
- A constructor is called depending upon the number and type of arguments passed.
- While creating the object, arguments must be passed to let compiler know, which constructor needs to be called.

```
#include <iostream>
using namespace std;

class Teacher
{
public:
    string name;
    string subject;
    string dept;
    double salary;

    Teacher(string n, string s, double sal)
    {
        name = n;
        subject = s;
        salary = sal;
    }
}
```

```
}  
Teacher(string n, string s){  
    name = n;  
    subject = s;  
}  
  
void getInfo()  
{  
    cout << "Name : " << name << endl;  
    cout << "Subject : " << subject << endl;  
}  
};  
  
int main()  
{  
    Teacher t1("Aadil", "C++");  
    Teacher t2("Adnan", "OOPS", 25000);  
    t1.getInfo();  
    t2.getInfo();  
    cout << "Salary : " << t2.salary << endl;  
    return 0;  
}
```

Destructors in C++

A destructor is also a special member function as a constructor. Destructor destroys the class objects created by the constructor.

Destructor has the same name as their class name preceded by a tilde (~) symbol. It is not possible to define more than one destructor.

The destructor is only one way to destroy the object created by the constructor. Hence destructor can-not be overloaded.

Destructor neither requires any argument nor returns any value. It is automatically called when the object goes out of scope.

Destructors release memory space occupied by the objects created by the constructor. In destructor, objects are destroyed in the reverse of object creation.

Syntax:

```
~className(){}
```

Example:

```
#include <iostream>
using namespace std;
class student
{
public:
    string name;
    double *cgpaPtr;

    student(string name, double cgpa)
    {
        this->name = name;
        cgpaPtr = new double;
        *cgpaPtr = cgpa;
    }

    ~student()
    {
        cout << "Destructor called" << endl;
        delete cgpaPtr;
    }
}
```

```
void getInfo()
{
    cout << "Name : " << name << endl;
    cout << "Cgpa : " << *cgpaPtr << endl;
}
};

int main()
{
    student s1("Aadil", 9.1);
    s1.getInfo();
}
```